1. What is a Static Class in C#?

A **static class** in C# is a class that **cannot be instantiated (no objects can be created)** and **contains only static members**.

That means:

- You **cannot create an object** of the class.
- All members inside the class **must be static**.
- Members are accessed **using the class name**, not through an object.

Syntax

```
public static class ClassName
{
    public static void MethodName()
    {
        // logic
    }
}
```

---

2. Example of Static Class

```
using System;

public static class Calculator
{
    public static int Add(int a, int b)
    {
        return a + b;
    }
```

```
    public static int Multiply(int a, int b)
    {
        return a * b;
    }
}

class Program
{
    static void Main()
    {
        int result1 = Calculator.Add(10, 20);
        int result2 = Calculator.Multiply(5, 4);

        Console.WriteLine(result1);
        Console.WriteLine(result2);
    }
}
```

Output

30

20

Important Observation

Notice this line:

Calculator.Add(10, 20);

We are **calling the method using the class name**, not using an object.

---

3. Key Characteristics of Static Classes

1️⃣ Cannot Create Objects

❌ This is not allowed:

```
Calculator obj = new Calculator();
```

Compiler error:
Cannot create an instance of the static class.

2️⃣ All Members Must Be Static

Inside a static class:

✓ Allowed

```
public static int Add(int a, int b)
{
    return a + b;
}
```

❌ Not allowed

```
public int Add(int a, int b)
{
    return a + b;
}
```

3️⃣ Static Classes Cannot Be Inherited

You **cannot derive a class from a static class**.

```
public static class A { }


class B : A   // ❌ Not allowed
{
```

}

4️⃣ Static Classes Are Automatically Sealed

A static class behaves like:

sealed class

Meaning:

- It **cannot be inherited**
- It **cannot be instantiated**

---

4. Real Example from .NET

A common static class in .NET is **Math**.

```
double result = Math.Sqrt(25);

Console.WriteLine(result);
```

Here:

- Math is a **static class**
- Sqrt() is a **static method**

No object is needed.

---

5. Static Constructor in Static Class

Static classes can have **static constructors**.

A static constructor:

- Executes **only once**
- Executes **automatically when the class is first used**

Example

```
using System;
```

```
public static class Logger
{
    static Logger()
    {
        Console.WriteLine("Logger initialized");
    }

    public static void Log(string message)
    {
        Console.WriteLine(message);
    }
}

class Program
{
    static void Main()
    {
        Logger.Log("Application Started");
    }
}
```

Output

Logger initialized

Application Started

The **static constructor runs first**.

---

6. When Should We Use Static Classes?

Use static classes when you want **utility/helper functionality** that does not require object state.

Common use cases:

✓ Utility classes
✓ Mathematical operations
✓ Helper methods
✓ Configuration access

Example:

```
public static class StringHelper
{
    public static string ToUpperCase(string text)
    {
        return text.ToUpper();
    }
}
```

Usage:

```
string result = StringHelper.ToUpperCase("hello");
```

---

7. Static Class vs Normal Class

Feature	Static Class	Normal Class
Object Creation	❌ Not allowed	✓ Allowed
Members	Only static	Static + Instance
Inheritance	❌ Not supported	✓ Supported
Access	Using class name	Using object